

GUÍA PASO A PASO · SESIÓN 4

Flutter y React Native en Visual Studio Code

Desde cero: instalar, configurar y ejecutar tu primera app en cada entorno, con cada comando que debes escribir en la terminal.

Asignatura	Diseño de Aplicaciones para Dispositivos Móviles	**Programa**	Ingeniería de Software
Docente	Ing. Kevin Torrez	**Fecha**	Octubre de 2026

Esta guía te lleva paso a paso a dejar listos Flutter y React Native en tu computador. No necesitas experiencia: cada paso te dice dónde hacer clic, qué escribir en la terminal y qué debes ver. Si algo no sale igual, revisa la tabla de problemas comunes al final.

Antes de empezar

- Android Studio instalado y funcionando, con el emulador creado (guía de la sesión 3).
- Conexión a internet: vas a descargar varios programas.
- Espacio libre en disco: al menos 10 GB.
- Un celular Android o iPhone (opcional), para ver la app de React Native con Expo Go.
- Usa siempre carpetas sin tildes, ñ ni espacios: C:\dev para Flutter y C:\Proyectos para tus proyectos.

¿Qué es cada cosa?

Antes de instalar, conviene saber qué es cada programa y para qué lo vamos a usar.

Herramienta	Qué es	Para qué la usamos hoy
Android Studio	El programa oficial de Google para crear apps de Android. Trae el editor, el Android SDK, el emulador y Gradle (la herramienta que arma la app). Se programa en Kotlin.	Ya lo instalaste en la sesión 3. Flutter usa su Android SDK y su emulador.
Flutter	Framework de Google, gratuito y de código abierto, para crear apps de Android, iPhone, web y escritorio con un solo código. Se programa en Dart y la pantalla se arma con widgets.	Instalar el Flutter SDK y crear tu primera app de Flutter.

Herramienta	Qué es	Para qué la usamos hoy
React Native	Framework de Meta, de código abierto, para crear apps de Android e iPhone con JavaScript o TypeScript. Usa los botones y textos propios de cada sistema.	Crear tu primera app de React Native.
Expo y Expo Go	Expo es un conjunto de herramientas que facilita crear y probar apps de React Native; la documentación oficial recomienda empezar con él. Expo Go es la app del celular que muestra tu proyecto mientras lo programas.	Crear el proyecto y verlo en el celular sin compilar nada.
Visual Studio Code	Editor de código gratuito de Microsoft. Con extensiones se adapta a cada lenguaje y trae una terminal integrada.	Editor para Flutter y React Native, y lugar donde escribes los comandos.
Git	Programa que guarda el historial de cambios del código. Flutter lo usa para descargarse y actualizarse.	Requisito de Flutter; más adelante, el control de versiones del proyecto.
Node.js, npm y npx	Node.js ejecuta JavaScript en el computador. npm descarga librerías y npx ejecuta herramientas sin instalarlas.	Requisito de React Native y Expo.
SDK	Kit de herramientas para crear apps de una plataforma.	Android SDK y Flutter SDK.
Terminal y PATH	La terminal es la ventana donde se escriben comandos. El PATH es la lista de carpetas donde Windows busca esos programas.	Todos los comandos de esta guía se escriben en la terminal de VS Code.

Mapa de navegación visual

1. **Preparar el Android SDK:** [Android Studio] → SDK Manager → SDK Tools → Android SDK Command-line Tools → Apply
2. **Instalar Git y VS Code:** git-scm.com → Download → Install · code.visualstudio.com → Download → Install · [Terminal] → git --version
3. **Instalar Flutter:** [VS Code] → Extensiones → Flutter → Install → Ctrl+Shift+P → Flutter: New Project → Download SDK → C:\dev → Add SDK to PATH
4. **Verificar:** [Terminal] → flutter --version → flutter doctor (y, solo si lo pide, flutter doctor --android-licenses)
5. **Crear el proyecto de Flutter:** [Terminal] → mkdir C:\Proyectos → cd C:\Proyectos → flutter create mi_app_flutter
6. **Ejecutar Flutter:** [Terminal] → flutter emulators → flutter emulators --launch … → flutter run → tecla r
7. **Instalar Node.js:** nodejs.org → LTS → Install → [Terminal] → node -v → npm -v
8. **Crear el proyecto de Expo:** [Terminal] → Set-ExecutionPolicy … → cd C:\Proyectos → npx create-expo-app@latest mi-app-rn
9. **Ver la app en el celular:** [Terminal] → npx expo start → [Expo Go] → escanear el código QR
10. **Cambiar el texto:** app/(tabs)/index.tsx → Welcome! → Ctrl+S

Parte A · Flutter

Paso 1: Preparar el Android SDK

¿Qué es? Las Command-line Tools son herramientas del Android SDK que se usan desde la terminal. Flutter las necesita para revisar el SDK y aceptar las licencias de Android.

Ruta: [Android Studio] → [Menú principal ☰] → Tools → SDK Manager → [Pestaña SDK Tools] → [Casilla Android SDK Command-line Tools (latest)] → Apply

1. Abre Android Studio. En la pantalla de bienvenida, haz clic en [More Actions / tres puntos ⋮] → SDK Manager. Si tienes un proyecto abierto: [Menú principal ☰] → Tools → SDK Manager.
2. Haz clic en [Pestaña «SDK Tools»], en la parte superior del panel.
3. Marca [Casilla «Android SDK Command-line Tools (latest)»]. Verifica que también estén marcadas Android SDK Build-Tools y Android SDK Platform-Tools.
4. Haz clic en [Botón Apply] y luego en [OK]. Espera la descarga y haz clic en [Finish].

Lo que debes ver: en la columna Status, las tres herramientas dicen *Installed*. Ya puedes cerrar Android Studio.

Paso 2: Instalar Git y Visual Studio Code

¿Qué es? Git guarda el historial de cambios del código y es el programa con el que Flutter se descarga y se actualiza. Visual Studio Code (VS Code) es el editor donde vas a escribir el código y los comandos.

Ruta: git-scm.com → [Download for Windows] → Next → Install · code.visualstudio.com → [Download for Windows] → Next → Install

1. Entra a git-scm.com y haz clic en [Botón «Download for Windows»]. Descarga el instalador de 64 bits («64-bit Git for Windows Setup»).
2. Abre el instalador. Hay muchas pantallas: en todas deja las opciones como vienen y haz clic en [Next]. Al final, [Install] y [Finish].
3. Entra a code.visualstudio.com y haz clic en [Botón «Download for Windows»].
4. Abre el instalador, acepta el acuerdo de licencia y, en «Tareas adicionales», deja marcada [Casilla «Agregar a PATH»]. Luego [Install] y [Finish].

Cómo usar la terminal de VS Code

En esta guía vas a escribir varios comandos. Todos se escriben en la **terminal** de VS Code, siempre de la misma forma:

1. Abre VS Code y haz clic en [Menú Terminal] → [New Terminal] (en español: Terminal → Nuevo terminal). Atajo: Ctrl+ñ.
2. En la parte inferior aparece la terminal, con una línea parecida a esta:

   La terminal está lista cuando ves:
   ```
   PS C:\Users\TuUsuario>
   ```

3. Haz clic dentro de la terminal, escribe el comando (o cópialo de la guía y pégalo con Ctrl+V) y presiona **Enter**.
4. Espera a que vuelva a aparecer la línea **PS C:\...>** antes de escribir el siguiente comando.

   Dos reglas de oro: si un comando «no se reconoce», cierra VS Code por completo y vuelve a abrirlo (después de instalar un programa, la terminal necesita reiniciarse). Para detener un comando que sigue corriendo, haz clic en la terminal y presiona **Ctrl+C**.

5. Comprueba que Git quedó instalado:

   Escribe en la terminal y presiona Enter:
   ```
   git --version
   ```

   Debes ver algo así:
   ```
   git version 2.xx.x.windows.1
   ```

Paso 3: Instalar Flutter desde VS Code

¿Qué es? El Flutter SDK es el kit de Flutter: trae el lenguaje Dart, el comando flutter y las piezas para crear apps. La extensión de Flutter conecta VS Code con ese kit y lo descarga por ti.

Ruta: [VS Code] → [Extensiones] → Flutter → [Install] → Ctrl+Shift+P → «Flutter: New Project» → [Download SDK] → C:\dev → [Clone Flutter] → [Add SDK to PATH]

1. En VS Code, haz clic en [Icono de Extensiones / cuatro cuadrados], en la barra lateral izquierda (atajo Ctrl+Shift+X).
2. Escribe Flutter en el buscador y haz clic en [Botón Install] de la extensión «Flutter» (de Dart Code). También se instala la extensión de Dart.
3. Abre la paleta de comandos con **Ctrl+Shift+P**, escribe flutter y elige [«Flutter: New Project»].
4. VS Code avisa que no encuentra el Flutter SDK. Haz clic en [Botón «Download SDK»].
5. En la ventana que se abre, entra a la unidad C:, crea la carpeta dev, selecciónala y haz clic en [Botón «Clone Flutter»].
6. Espera: abajo a la derecha verás que se está descargando e iniciando el Flutter SDK. Puede tardar de 5 a 15 minutos.
7. Cuando VS Code pregunte si quieres agregar Flutter al PATH, haz clic en [Botón «Add SDK to PATH»].
8. Cierra VS Code por completo y vuelve a abrirlo.

Ojo con la carpeta: si tu usuario de Windows tiene tildes o espacios (por ejemplo, «José Pérez»), no instales Flutter dentro de la carpeta de tu usuario: usa C:\dev.

Paso 4: Verificar con flutter doctor

¿Qué es? flutter doctor es un comando que revisa la instalación y te dice qué está bien y qué falta, como un chequeo médico. Lo vas a usar cada vez que algo no funcione.

1. Abre una terminal nueva ([Menú Terminal] → [New Terminal]).
2. Comprueba que Flutter responde. La primera vez puede tardar uno o dos minutos:

Escribe en la terminal y presiona Enter:

```
flutter --version
```

Debes ver algo así:

```
Flutter 3.xx.x • channel stable • https://github.com/flutter/flutter.git
Tools • Dart 3.x.x • DevTools 2.x.x
```

3. Haz el chequeo completo. La primera vez, antes del resultado, verás que descarga algunos componentes ([1/10] Material Fonts...): es normal.

Escribe en la terminal y presiona Enter:

```
flutter doctor
```

Debes ver algo así:

```
Doctor summary (to see all details, run flutter doctor -v):
[√] Flutter (Channel stable, 3.47.x, on Microsoft Windows ..., locale es-CO)
[√] Windows Version
[√] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[√] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
[√] Connected device (3 available)
[!] Network resources
    X A network error occurred while checking "https://cocoapods.org/"
```

4. Si en Android toolchain aparece «cmdline-tools component is missing», te faltó el paso 1: hazlo y vuelve a escribir flutter doctor.
5. Si aparece «Android license status unknown», escribe:

Escribe en la terminal y presiona Enter:

```
flutter doctor --android-licenses
```

Con las herramientas nuevas de Android es normal que responda esto, sin preguntarte nada:

Respuesta normal:

```
WARNING: The SDK Manager CLI tool (sdkmanager) is deprecated.
Android CLI will be used instead.
...
Warning: The --licenses option is no longer needed.
```

No pasa nada: las licencias ya se aceptaron cuando instalaste Android Studio. Vuelve a escribir flutter doctor.

Lo que debes ver: [√] en Flutter y en Android toolchain. Visual Studio (apps de escritorio para Windows) y Network resources (cocoapods.org, que solo se usa para apps de iPhone en Mac) pueden salir con [X] o [!]: no importan para crear apps de Android.

Paso 5: Crear tu primer proyecto de Flutter

¿Qué es? Un proyecto es la carpeta con todo el código y la configuración de una app. El comando **flutter create** crea esa carpeta con una app de ejemplo lista para ejecutar; el código principal queda en lib/main.dart.

1. Crea la carpeta donde vas a guardar tus proyectos:

Escribe en la terminal y presiona Enter:

```
mkdir C:\Proyectos
```

Si te dice que «ya existe un elemento con el nombre especificado», no pasa nada: la carpeta ya estaba creada. Sigue con el punto 2.

2. Entra a la carpeta:

Escribe en la terminal y presiona Enter:

```
cd C:\Proyectos
```

Debes ver algo así:

```
PS C:\Proyectos>
```

3. Crea el proyecto. El nombre va en minúsculas y con guion bajo: Flutter no acepta espacios ni mayúsculas.

Escribe en la terminal y presiona Enter:

```
flutter create mi_app_flutter
```

Debes ver algo así:

```
All done!
In order to run your application, type:

  $ cd mi_app_flutter
  $ flutter run
```

4. Abre el proyecto en VS Code: [Menú File] → [Open Folder] (en español: Archivo → Abrir carpeta) → C:\Proyectos\mi_app_flutter → [Botón «Seleccionar carpeta»].
5. Si VS Code pregunta si confías en los autores de los archivos, haz clic en [«Yes, I trust the authors»] (Sí, confío en los autores).
6. Abre una terminal nueva. Ahora la línea muestra la carpeta del proyecto:

Debes ver algo así:

```
PS C:\Proyectos\mi_app_flutter>
```

Paso 6: Encender el emulador y ejecutar la app

¿Qué es? El emulador es un celular virtual dentro del computador. **flutter run** convierte tu código en una app (compila), la instala en el emulador y la abre. Con **hot reload** (tecla r) ves tus cambios en un segundo, sin volver a compilar.

1. Mira qué emuladores tienes:

Escribe en la terminal y presiona Enter:

```
flutter emulators
```

Debes ver algo así:

```
1 available emulator:

Medium_Phone_API_36 • Medium Phone API 36 • Generic • android

To run an emulator, run 'flutter emulators --launch <emulator id>'.
```

La primera palabra de la línea (en el ejemplo, **Medium_Phone_API_36**) es el nombre de tu emulador. El tuyo puede llamarse distinto.

2. Enciende el emulador. Escribe el comando cambiando el nombre del ejemplo por el que te salió en la lista:

Escribe en la terminal y presiona Enter:

```
flutter emulators --launch Medium_Phone_API_36
```

> **Ojo:** Medium_Phone_API_36 es solo un ejemplo. Si lo copias tal cual y tu emulador se llama distinto, verás «No emulator found that matches». Escribe el nombre exacto de tu lista, por ejemplo `flutter emulators --launch Pixel_9_Pro_XL`.

Se abre una ventana con el celular virtual. Espera a que muestre la pantalla de inicio: tarda de 1 a 3 minutos.

> También puedes encenderlo como en la sesión 3: Android Studio → Device Manager → Play .

3. Comprueba que Flutter ve el emulador. Busca la línea que dice **emulator-5554**:

Escribe en la terminal y presiona Enter:

```
flutter devices
```

Debes ver algo así:

```
sdk gphone64 x86 64 (mobile) • emulator-5554 • android-x64    • Android ... (emulator)
Windows (desktop)            • windows       • windows-x64    • Microsoft Windows ...
Chrome (web)                 • chrome        • web-javascript • Google Chrome ...
```

4. Verifica que la terminal esté en la carpeta del proyecto (debe decir **PS C:\Proyectos\mi_app_flutter>**). Si dice otra cosa, escribe `cd C:\Proyectos\mi_app_flutter`. Luego ejecuta la app:

Escribe en la terminal y presiona Enter:

```
flutter run
```

Si aparece una lista de dispositivos, escribe el número del que dice **emulator-5554** y presiona Enter:

Debes ver algo así:

```
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: sdk gphone64 x86 64 (emulator-5554)
Please choose one (or "q" to quit): 3
```

5. Espera. La primera vez tarda varios minutos, porque Gradle prepara todo. Cuando termine verás:

Debes ver algo así:

```
√ Built build\app\outputs\flutter-apk\app-debug.apk
...
Flutter run key commands.
r Hot reload.
R Hot restart.
q Quit (terminate the application on the device).
```

> **Lo que debes ver:** en el emulador, una app con un contador y un botón **+**. Tócalo: el número aumenta.

Si sale «UnknownHostException: services.gradle.org»: la red no deja descargar Gradle. Conéctate a otra red (por ejemplo, los datos de tu celular) y vuelve a escribir flutter run: solo la primera compilación lo necesita. Mientras tanto, puedes ver tu app en Chrome con `flutter run -d chrome`.

Paso 6 (continuación): Poner tu nombre en la app

6. Sin cerrar la terminal, abre el archivo **lib/main.dart**: en el panel izquierdo (Explorer), abre la carpeta lib y haz clic en main.dart.
7. Busca esta línea. Para encontrarla rápido, presiona Ctrl+F y escribe Flutter Demo:

Así está:

```
home: const MyHomePage(title: 'Flutter Demo Home Page'),
```

8. Cambia el texto entre comillas por tu nombre y guarda con **Ctrl+S**:

Así debe quedar (con tu nombre):

```
home: const MyHomePage(title: 'Hola, soy Ana'),
```

9. Haz clic dentro de la terminal y presiona la tecla **r** (hot reload):

Debes ver algo así:

```
Performing hot reload...
Reloaded 1 of 700 libraries in 450ms.
```

Lo que debes ver: la barra superior de la app dice «Hola, soy Ana». Toma la captura de pantalla para tu evidencia.

10. Para detener la app, haz clic en la terminal y presiona la tecla **q**.

Parte B · React Native con Expo

Paso 7: Instalar Node.js

¿Qué es? Node.js ejecuta JavaScript en tu computador (fuera del navegador). Trae **npm**, que descarga librerías, y **npx**, que ejecuta herramientas como Expo. React Native y Expo no funcionan sin él.

Ruta: nodejs.org → [Botón LTS] → [Instalador .msi] → Next → Install → [Terminal] → node -v → npm -v

1. Entra a nodejs.org y descarga la versión **LTS** (la recomendada para la mayoría de usuarios).
2. Abre el instalador .msi y avanza con [Next], aceptando la licencia y las opciones que vienen marcadas.
3. En la pantalla «Tools for Native Modules», deja la casilla **sin marcar**: no la necesitas.
4. Haz clic en [Install] y luego en [Finish].
5. Cierra VS Code por completo y vuelve a abrirlo, para que la terminal reconozca Node.js.
6. Abre una terminal nueva y comprueba la instalación:

Escribe en la terminal y presiona Enter:

```
node -v
```

Debes ver algo así:

```
v24.x.x
```

Escribe en la terminal y presiona Enter:

```
npm -v
```

Debes ver algo así:

```
11.x.x
```

Lo que debes ver: dos números de versión (pueden ser más altos que los del ejemplo).

Paso 8: Crear tu proyecto de Expo

¿Qué es? Expo facilita crear y probar apps de React Native. **create-expo-app** crea el proyecto con una app de ejemplo. El comando Set-ExecutionPolicy cambia una regla de seguridad de Windows que impide ejecutar scripts; se cambia solo para tu usuario.

1. En tu celular, instala la app **Expo Go** desde Play Store o App Store.
2. Permite que la terminal ejecute las herramientas de Node.js. Se hace una sola vez:

   Escribe en la terminal y presiona Enter:
   ```
   Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
   ```

   Te preguntará si quieres cambiar la directiva de ejecución. Escribe **S** y presiona Enter (si tu Windows está en inglés, escribe **Y**).

   Este paso evita el error «no se puede cargar el archivo npx.ps1 porque la ejecución de scripts está deshabilitada».

3. Entra a la carpeta de tus proyectos:

   Escribe en la terminal y presiona Enter:
   ```
   cd C:\Proyectos
   ```

4. Crea el proyecto. Escribe el comando tal cual, sin agregar nada más:

   Escribe en la terminal y presiona Enter:
   ```
   npx create-expo-app@latest mi-app-rn
   ```

   Si pregunta «Ok to proceed? (y)», escribe **y** y presiona Enter. Espera varios minutos hasta ver:

   Debes ver algo así:
   ```
   Your project is ready!

   To run your project, navigate to the directory and run one of the following npm commands.

   - cd mi-app-rn
   - npm run android
   - npm run ios
   - npm run web
   ```

5. Abre el proyecto en VS Code: [Menú File] → [Open Folder] → C:\Proyectos\mi-app-rn → [Botón «Seleccionar carpeta»] → [«Yes, I trust the authors»].
6. Abre una terminal nueva. Debe mostrar:

   Debes ver algo así:
   ```
   PS C:\Proyectos\mi-app-rn>
   ```

Paso 9: Ver la app en tu celular

¿Qué es? npx expo start enciende un servidor en tu computador que envía la app al celular por la red Wi-Fi. Expo Go la recibe y la muestra, sin compilar ni instalar nada.

1. Conecta el celular a la **misma red Wi-Fi** que el computador.
2. Inicia Expo:

Escribe en la terminal y presiona Enter:

```
npx expo start
```

Debes ver un código QR y, debajo, algo así:

```
› Metro waiting on exp://192.168.x.x:8081
› Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

› Press a │ open Android
› Press r │ reload app
```

3. En Android, abre Expo Go y toca [«Scan QR code»]. En iPhone, abre la app Cámara y apunta al código.
4. La app se abre en tu celular con el texto «Welcome!».

Sin celular: enciende el emulador (paso 6, punto 2), haz clic en la terminal y presiona la tecla **a**. Expo instala Expo Go en el emulador y abre la app.

Si el celular no se conecta: haz clic en la terminal, presiona Ctrl+C y escribe `npx expo start --tunnel`. La primera vez pregunta si quieres instalar un complemento: escribe **y** y presiona Enter.

Paso 10: Poner tu nombre en la app

¿Qué es? El código de la app está en archivos .tsx (TypeScript con React). Cada vez que guardas un cambio, Expo lo envía al celular automáticamente: eso se llama Fast Refresh.

1. En el panel izquierdo de VS Code, abre la carpeta **app**, luego **(tabs)**, y haz clic en **index.tsx**.
2. Busca el texto Welcome! (presiona Ctrl+F y escríbelo). Verás una línea parecida a esta:

Así está:

```
<ThemedText type="title">Welcome!</ThemedText>
```

3. Cambia Welcome! por tu saludo y guarda con **Ctrl+S**:

Así debe quedar (con tu nombre):

```
<ThemedText type="title">¡Hola, soy Ana!</ThemedText>
```

4. La app del celular se actualiza sola. Si no cambia, haz clic en la terminal y presiona la tecla **r**.

Lo que debes ver: «¡Hola, soy Ana!» en la app. Toma la captura de pantalla para tu evidencia.

5. Para detener Expo, haz clic en la terminal y presiona **Ctrl+C**.

Problemas comunes

Lo que ves	Causa probable	Solución
«flutter», «git» o «node» no se reconoce como un comando	La terminal se abrió antes de instalar el programa	Cierra VS Code por completo y vuelve a abrirlo. Si sigue con Flutter, repite «Add SDK to PATH» (paso 3).
flutter doctor: «cmdline-tools component is missing»	Faltan las Command-line Tools del Android SDK	Haz el paso 1: SDK Manager → SDK Tools → Android SDK Command-line Tools.
flutter doctor --android-licenses responde «The --licenses option is no longer needed»	Las herramientas nuevas de Android ya no piden aceptar licencias en la terminal	Es normal: vuelve a escribir `flutter doctor`.
flutter doctor sigue diciendo «Android license status unknown»	Flutter desactualizado o falta la carpeta de licencias	Escribe `flutter upgrade`. Si sigue, en el SDK Manager de Android Studio instala o actualiza un componente y acepta su licencia.
[!] Network resources: cocoapods.org	La red no deja consultar ese sitio	Ignóralo: CocoaPods solo se usa para apps de iPhone en Mac.
mkdir: «ya existe un elemento con el nombre especificado»	La carpeta ya estaba creada	No pasa nada: sigue con `cd C:\Proyectos`.
flutter run: «No supported devices connected»	El emulador no está encendido	Enciéndelo con `flutter emulators --launch ...` (paso 6) y espera la pantalla de inicio.
«No emulator found that matches»	Escribiste el nombre del ejemplo o uno distinto al de tu lista	Escribe `flutter emulators`, copia la primera palabra de la línea de tu emulador y úsala en --launch.
flutter run: «No pubspec.yaml file found»	La terminal no está en la carpeta del proyecto	Escribe `cd C:\Proyectos\mi_app_flutter` y vuelve a escribir flutter run.
flutter run: «UnknownHostException: services.gradle.org» y «Gradle task assembleDebug failed»	La red bloquea la descarga de Gradle (la primera compilación necesita internet sin bloqueos)	Conéctate a otra red (por ejemplo, los datos del celular) y repite flutter run; las siguientes compilaciones funcionan sin esa red. Mientras tanto: `flutter run -d chrome`.
Presiono r y no pasa nada	El cursor no estaba en la terminal	Haz clic dentro de la terminal y vuelve a presionar r.
«No se puede cargar el archivo npx.ps1 porque la ejecución de scripts está deshabilitada»	Política de seguridad de PowerShell	Haz el punto 2 del paso 8 (`Set-ExecutionPolicy`).

Lo que ves	Causa probable	Solución
Set-ExecutionPolicy dice «acceso denegado»	El computador tiene restricciones	En la terminal, abre la flecha junto al botón + y elige «Command Prompt»: ahí npx funciona sin ese cambio.
Expo Go no abre la app o se queda cargando	El celular y el computador están en redes distintas, o la red los aísla	Usa la misma Wi-Fi; si no funciona, `npx expo start --tunnel` o la tecla a con el emulador.
Expo Go dice que el proyecto no es compatible con su versión	El proyecto se creó con una versión de Expo distinta a la de Expo Go	Actualiza Expo Go y crea el proyecto con el comando del paso 8, sin agregar nada.
La primera ejecución de Flutter tarda mucho	Gradle descarga y prepara todo la primera vez	Es normal: espera. Las siguientes ejecuciones tardan segundos.

Mini-glosario

Término	Qué es	Ejemplo
Terminal	Ventana donde se escriben comandos.	Terminal de VS Code
Comando	Instrucción que se escribe en la terminal y se ejecuta con Enter.	`flutter doctor`
cd	Comando para entrar a una carpeta.	`cd C:\Proyectos`
mkdir	Comando para crear una carpeta.	`mkdir C:\Proyectos`
SDK	Kit de herramientas para crear apps de una plataforma.	Android SDK, Flutter SDK
PATH	Lista de carpetas donde Windows busca los programas que escribes en la terminal.	Add SDK to PATH
Emulador	Celular virtual que corre en el computador.	`flutter emulators`
Node.js, npm y npx	Node.js ejecuta JavaScript en el computador; npm descarga librerías; npx ejecuta herramientas.	`npx expo start`
Expo Go	App del celular que muestra tu app de React Native mientras la programas.	Escanear el código QR
Hot reload	Ver los cambios en la app sin volver a compilar.	Tecla r en la terminal

Anexo · Formato de entrega de evidencia técnica

Entrega individual: cada estudiante muestra su propio computador. Un documento PDF con las capturas, subido al aula virtual antes del [dd/mm/2026].

Nombre		**Equipo**	
Fecha		**Computador**	Sistema: _____ RAM: ____ GB

1. Versiones instaladas

Herramienta	Comando o lugar para verla	Versión
Android Studio	[Menú principal ☰] → Help → About	
Flutter	`flutter --version`	
Node.js	`node -v`	
npm	`npm -v`	
Git	`git --version`	

2. Capturas de pantalla

☐ Android Studio: la app «Hello [tu nombre]!» en el emulador o en el celular (sesión 3).

☐ El resultado de flutter doctor, con Flutter y Android toolchain en [√].

☐ La app de Flutter con tu nombre en la barra superior.

☐ La app de Expo con tu nombre en el celular o en el emulador.

3. Un problema que tuve y cómo lo resolví

4. ¿Qué entorno me pareció más fácil y cuál usaría mi equipo para el proyecto? ¿Por qué?

Cómo se califica

Criterio	Peso	Qué se evalúa
Conocimiento	30 %	Explica para qué sirve cada herramienta instalada.
Habilidad	30 %	Entornos instalados y verificados: versiones y flutter doctor.
Acción	40 %	Las tres apps ejecutadas con su nombre, o problemas resueltos y documentados.
Actitud	Factor	Puntualidad, ayuda a los compañeros y tolerancia al error. Multiplica la nota.

Nota = (0,30 × Conocimiento + 0,30 × Habilidad + 0,40 × Acción) × Factor de actitud. Si tu computador no permite instalar algún entorno, documenta el problema y lo que intentaste: también cuenta como evidencia.